Mushrooms Classification Project

Giriş

Çeşitli türlerin keşfi, özellikle doğanın karmaşık dokusunda yer alan mantarlar gibi, bilim insanlarını ve meraklıları büyüler. Bu veri seti, gilled mantarların karmaşık dünyasına iner, 1981'de Kuzey Amerika Mantarlarına Dair Audubon Derneği Alan Kılavuzu'ndan alınan 23 hayali örneği sunar. Bu örnekler, Agaricus ve Lepiota mantar ailesine ait türleri temsil eder, her biri kendine özgü özelliklere ve gizemlere sahiptir.

Mantarların yenilebilirliğini anlamak karmaşık bir bulmacadır. Zehirli Meşe ve Salkımlı Yapraklı Meyve Ağaçları için "üç yapraklı ise, bırak git" gibi basit bir özdeyiğin aksine, bir mantarın güvenli mi yoksa potansiyel olarak zararlı mı olduğunu belirlemek basit kurallara uymaz. Kılavuz, mantarın yenilebilirliğini ayırt etmek için evrensel bir kılavuzun olmadığını açıkça belirtir, bu da bu araştırmanın karmaşıklığını vurgular.

Proje Açıklaması

Bu proje temelde, sınıflandırma alanında çerçevelenen denetimli bir makine öğrenimi girişimidir. Temel amaç açıktır: belirgin özelliklere dayanarak mantarların yenilebilirliğini tahmin edebilen sağlam bir model geliştirmek. Bu çalışmanın bağlamında, hedef değer mantarları sınıflandırmak etrafında döner, özellikle onları kesinlikle yenilebilir, kesinlikle zehirli veya bilinmeyen yenilebilirlik ve tavsiye edilmeyen olarak tanımlar. Özellikle, sonraki sınıf, mantar toplama ve tüketiminde sıklıkla benimsenen dikkatli yaklaşımla zehirli kategoriyle birleştirilir.

Karşımızdaki karmaşık görev, çeşitli makine öğrenimi tekniklerinin ve algoritmalarının kapsamlı bir keşfini gerektirir. Sınıflandırma algoritmalarının gücünden yararlanarak, mantar veri setinde gizlenmiş karmaşık desenleri ortaya çıkarmak, nihayetinde mantar tüketimi ve güvenliği konusunda bilinçli kararlar için yol açmak amaçlanır.

Proje Amaçları

Mantarların dünyasında bu yolculuğa çıktığımızda, defterin genel hedefi açıkça ortaya çıkar: mantar özelliklerinin labirentinde gezinebilen, yenilebilirliğini kesinlikle ayırt edebilen bir tahmin modeli oluşturmak. Özellikleri titizlikle analiz ederek ve mühendislik yaparak, çeşitli makine öğrenimi algoritmalarını keşfederek ve model parametrelerini ayarlayarak, güvenilir bir mantar yenilebilirliği tahmin etme çerçevesi oluşturmak amaçlanır.

Bu keşif yoluyla, sadece gerçek dünya zorluklarını ele almak için makine öğreniminin potansiyelini kullanmakla kalmayıp aynı zamanda mantar taksonomisi ve güvenliği konusundaki genel anlayışa da katkıda bulunulur. Veri setine daha derinlemesine inerken ve sırlarını çözerken, bireyleri, mantarların büyüleyici ancak esrarengiz dünyasını özgüven ve netlikle gezinmek için gereken bilgi ve araçlarla donatmayı amaçlanır.

Değişkenler

cap-shape: bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s

cap-surface: fibrous=f,grooves=g,scaly=y,smooth=s

cap-color: brown=n,buff=b,cinnamon=c,gray=g,green=r,pink=p,purple=u,red=e,white=w,yellow=y

bruises: bruises=t,no=f

odor: almond=a,anise=l,creosote=c,fishy=y,foul=f,musty=m,none=n,pungent=p,spicy=s

gill-attachment: attached=a,descending=d,free=f,notched=n

gill-spacing: close=c,crowded=w,distant=d

gill-size: broad=b,narrow=n

gill-color: black=k,brown=n,buff=b,chocolate=h,gray=g,
green=r,orange=o,pink=p,purple=u,red=e,white=w,yellow=y

stalk-shape: enlarging=e,tapering=t

stalk-root: bulbous=b,club=c,cup=u,equal=e,rhizomorphs=z,rooted=r,missing=?

stalk-surface-above-ring: fibrous=f,scaly=y,silky=k,smooth=s

stalk-surface-below-ring: fibrous=f,scaly=y,silky=k,smooth=s

stalk-color-above-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o,pink=p,red=e,white=w,yellow=y

stalk-color-below-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o,pink=p,red=e,white=w,yellow=y

veil-type: partial=p,universal=u

veil-color: brown=n,orange=o,white=w,yellow=y

ring-number: none=n,one=o,two=t

ring-type: cobwebby=c,evanescent=e,flaring=f,large=l,none=n,pendant=p,sheathing=s,zone=z

spore-print-color:
black=k,brown=n,buff=b,chocolate=h,green=r,orange=o,purple=u,white=w,yellow=y

population: abundant=a,clustered=c,numerous=n,scattered=s,several=v,solitary=y

habitat: grasses=g,leaves=l,meadows=m,paths=p,urban=u,waste=w,woods=d

Projemizde bu kategorik değişkenleri kullanıp, mantarlarımızın zehirli olup olmadığının modellemesini yapacağız.

```
df.shape
```

```
(8124, 23)
```

Verimiz 23 değişken, 8124 gözlemden oluşmaktadır.

```
df.describe()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color
count	8124	8124	8124	8124	8124	8124	8124	8124	8124	8124	...	8124	8124	8124	8124	8124	8124	8124	8124
unique	2	6	4	10	2	9	2	2	2	12	...	4	9	9	1	4	3	5	9
top	e	x	y	n	f	n	f	c	b	b	...	s	w	w	p	w	o	p	w
freq	4208	3656	3244	2284	4748	3528	7914	6812	5612	1728	...	4936	4464	4384	8124	7924	7488	3968	2388

Verimiz ile alakalı özet istatistikler yukarıdaki görselde verilmiştir.

Gill-attachment, veil-type, veil-color, ring-number gibi değişkenlerdeki değerlerin adetlerinin dengesiz dağıldığı görülmektedir. İlerleyen kısımda görselleştirmeler ile bunu daha iyi bir biçimde göreceğiz.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   class                     8124 non-null   object
 1   cap-shape                 8124 non-null   object
 2   cap-surface               8124 non-null   object
 3   cap-color                 8124 non-null   object
 4   bruises                   8124 non-null   object
 5   odor                      8124 non-null   object
 6   gill-attachment           8124 non-null   object
 7   gill-spacing              8124 non-null   object
 8   gill-size                 8124 non-null   object
 9   gill-color                8124 non-null   object
 10  stalk-shape               8124 non-null   object
 11  stalk-root                8124 non-null   object
 12  stalk-surface-above-ring  8124 non-null   object
 13  stalk-surface-below-ring  8124 non-null   object
 14  stalk-color-above-ring    8124 non-null   object
 15  stalk-color-below-ring    8124 non-null   object
 16  veil-type                 8124 non-null   object
 17  veil-color                8124 non-null   object
 18  ring-number               8124 non-null   object
 19  ring-type                 8124 non-null   object
 20  spore-print-color         8124 non-null   object
 21  population                8124 non-null   object
 22  habitat                   8124 non-null   object
dtypes: object(23)
memory usage: 1.4+ MB
```

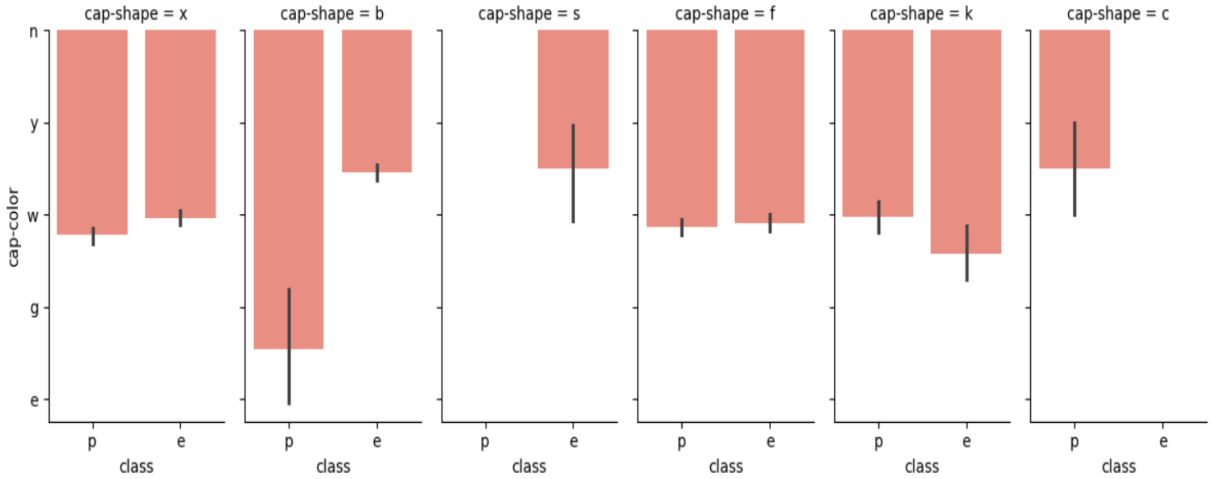
Verimiz tamamen kategorik değişkenlerden oluşup, boş gözlem bulundurmamaktadır.

```
df['stalk-root'].unique()
```

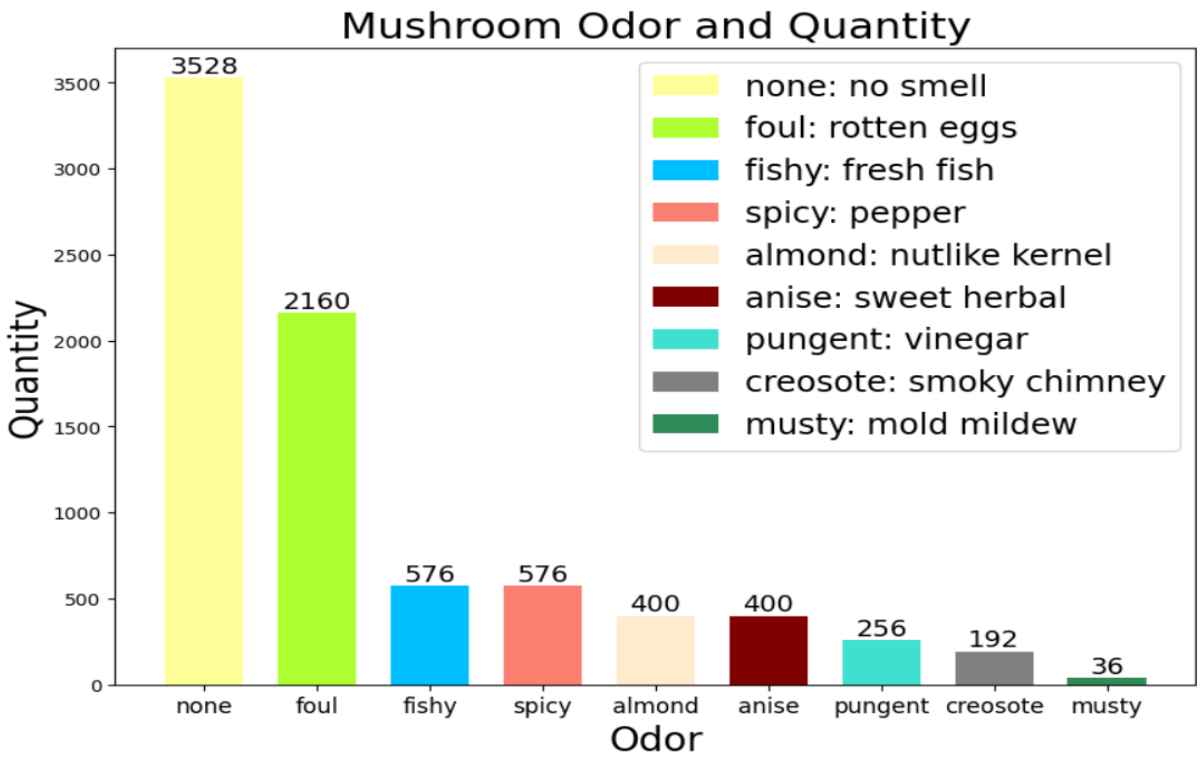
```
array(['e', 'c', 'b', 'r', '?'], dtype=object)
```

Ancak stalk-root değişkeninde ? şeklinde boş gözlemimiz bulunmaktadır. Bu değerlerin doldurulmasında random forest algoritması temelli imputer olan MissForest ile dolduracağız.

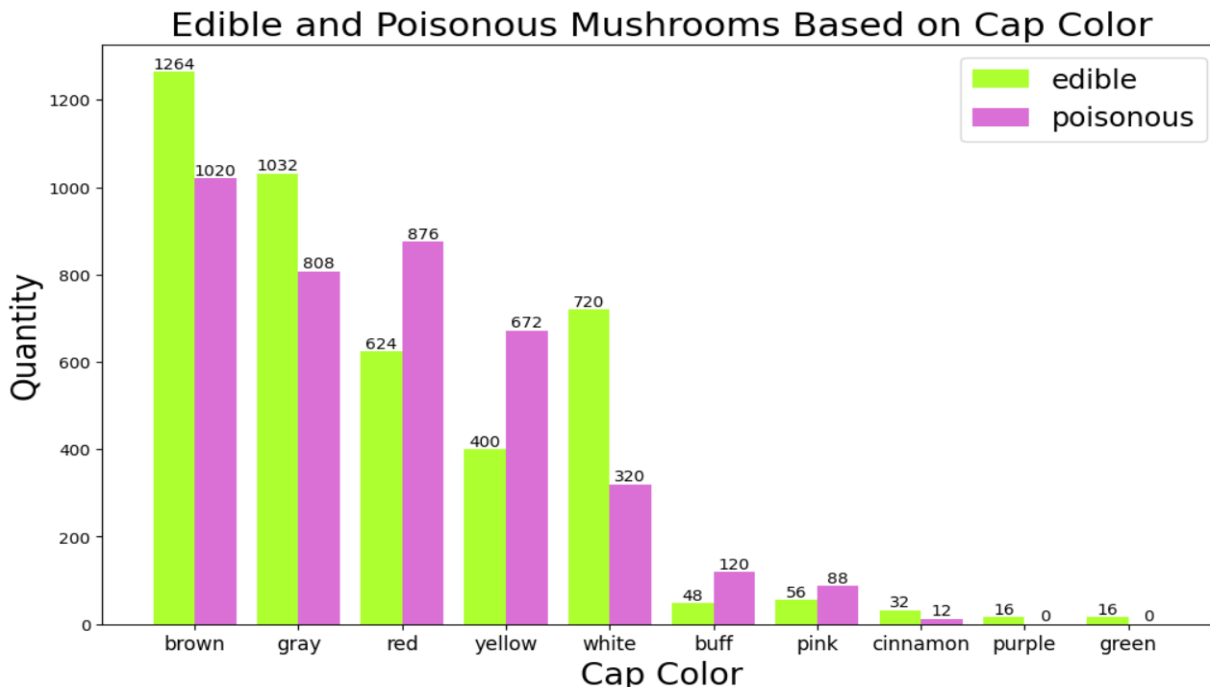
(görsel gelecek)



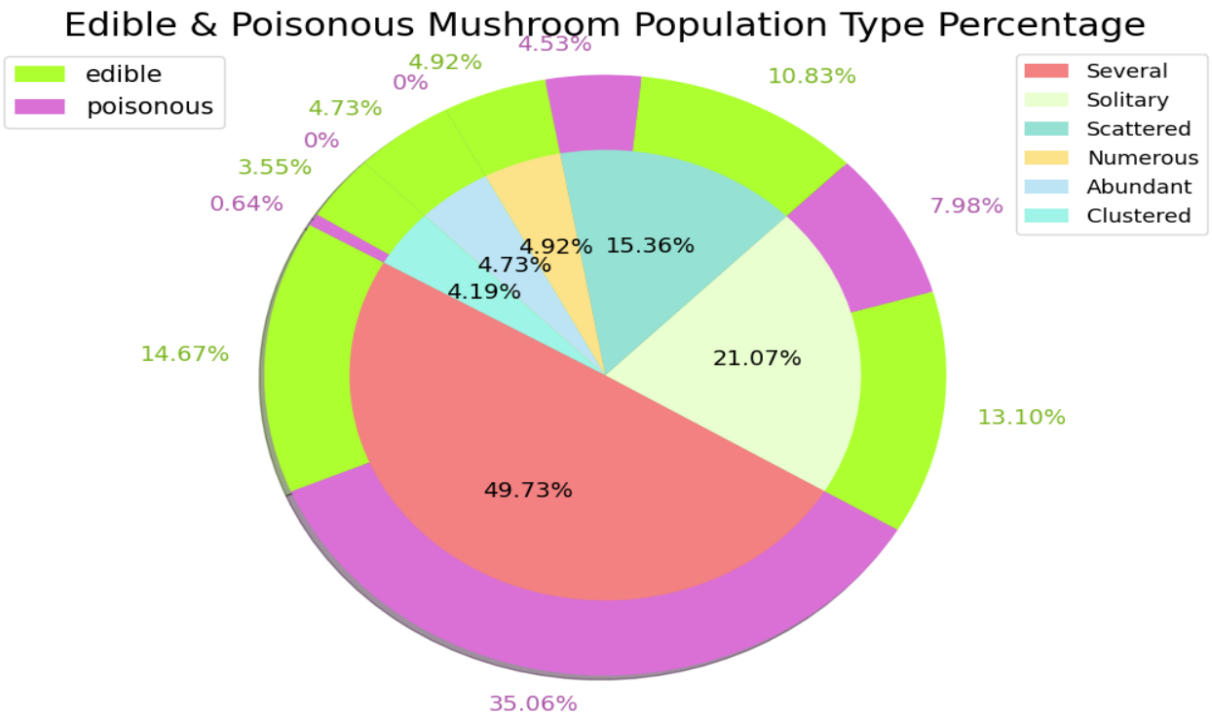
Kahve(n), sarı(y) ve beyaz(w) renklerdeki bombeli(x), toplu(k) ve düz(f) mantarlar genellikle görsel olarak benzerlik gösterir ve ayırt etmeleri zor olabilir. Ancak, çan(b) şeklindeki mantarlarda gri(g) ve kırmızı(e) renklerde olması, zehirli olma ihtimalini artırabilir. Öte yandan, koni(c) şeklindeki ve yine kahve, sarı, beyaz renklerde olan mantarlar genellikle zehirli olabilirler. Çukurlu(s) kahve, sarı ve beyaz renklerdeki mantarlar ise genellikle yenilebilir olarak kabul edilir. Bu nedenle, mantarları tanıma sürecinde renk ve şekil gibi görsel özelliklere dikkat etmek önemlidir.



Mantarların çoğunluğu kokusuz(none) ve kötü(foul) kokuya sahiptir.



Mantarların şapka renklerinin yenilebilir mi zehirli mi dağılımına baktığımızda. Kahve, gri, beyaz renkteki mantarların yenilebilir olma ihtimali daha yüksektir. Diğer yandan sarı, kırmızı mantarların ise zehirli olma ihtimali yüksektir yorumunda bulunabiliriz.



Mantarların neredeyse yarısı 'several' popülasyonuna aittir. Bu popülasyonun %35'i zehirli mantarlardan oluşmaktadır. Bu durum, zehirli mantarların yaygın olduğunu ve toplam popülasyon içinde önemli bir risk oluşturduğunu göstermektedir.

```
df['stalk-root'] = df['stalk-root'].replace('?', np.nan)

# Eksik verilere sahip ve olmayan örnekleri ayırma
missing_data = df[df['stalk-root'].isnull()]
available_data = df.dropna(subset=['stalk-root'])

# One-Hot Encoding işlemi
X = pd.get_dummies(available_data.drop('stalk-root', axis=1))
y = available_data['stalk-root']

# Eğitim ve test verisi olarak ayırma
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Rastgele orman modeli oluşturma
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Tahmin yapma
y_pred = model.predict(X_test)

# Modelin doğruluğunu değerlendirme
accuracy = accuracy_score(y_test, y_pred)
print("Model Doğruluğu:", accuracy)
```

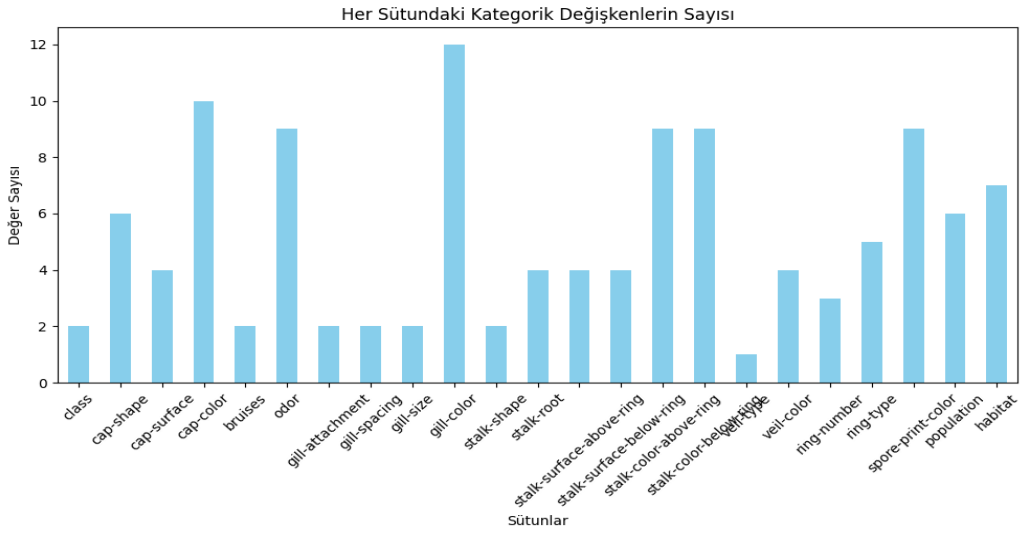
```
Model Doğruluğu: 1.0
```

```
print(completed_data['stalk-root'].unique())
df=completed_data
```

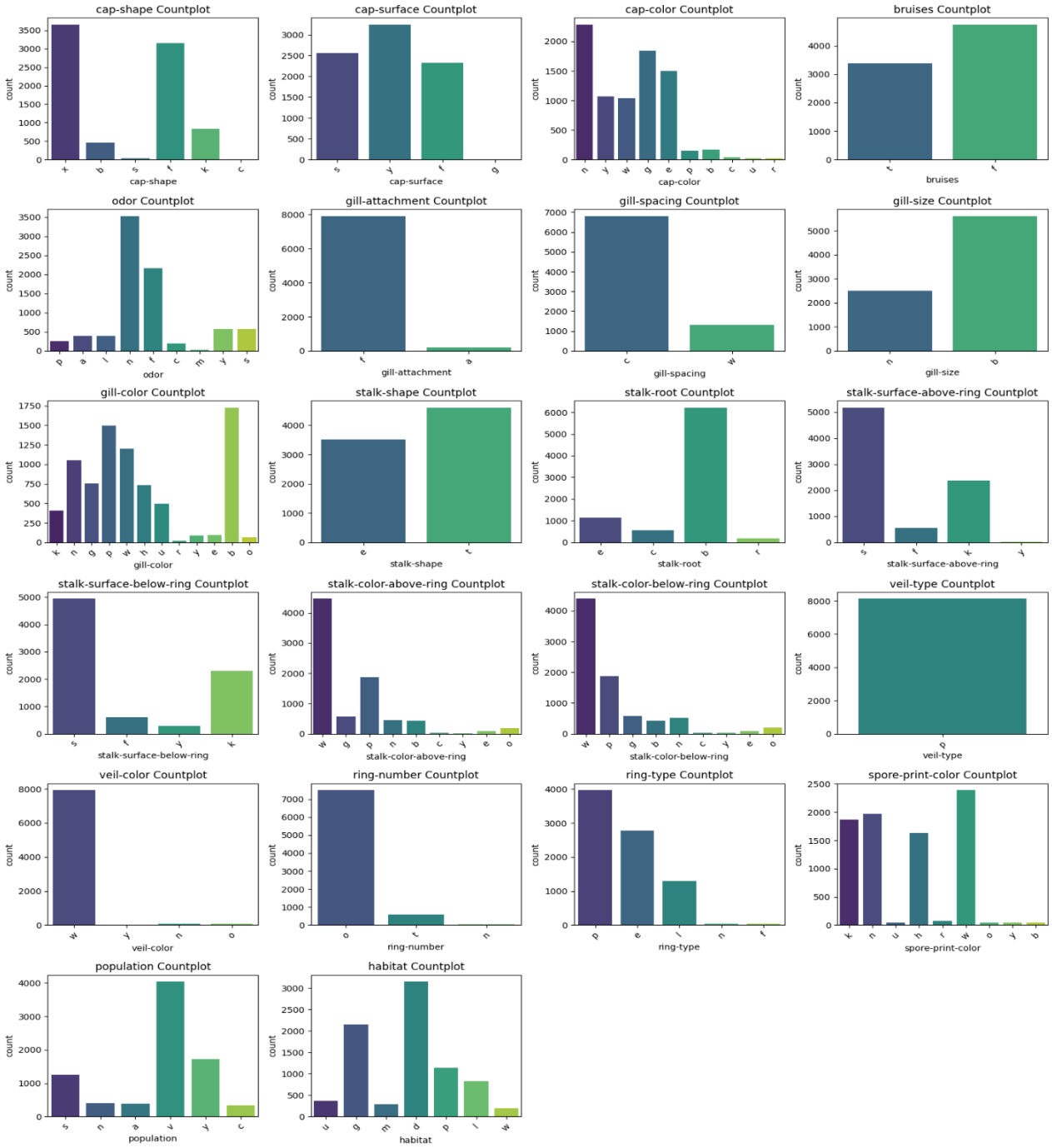
```
['e' 'c' 'b' 'r']
```

Görüldüğü üzere artık ? işaretli değerlerimiz, randomforrest algoritmasıyla dolduruldu . (Kaggle vs diğer platformlarda bu değerlerin random forest ile doldurulduğu görülmemiştir.)

Şimdi verilerimizi görselleştirip çıkarımlarımızı yapacağız.



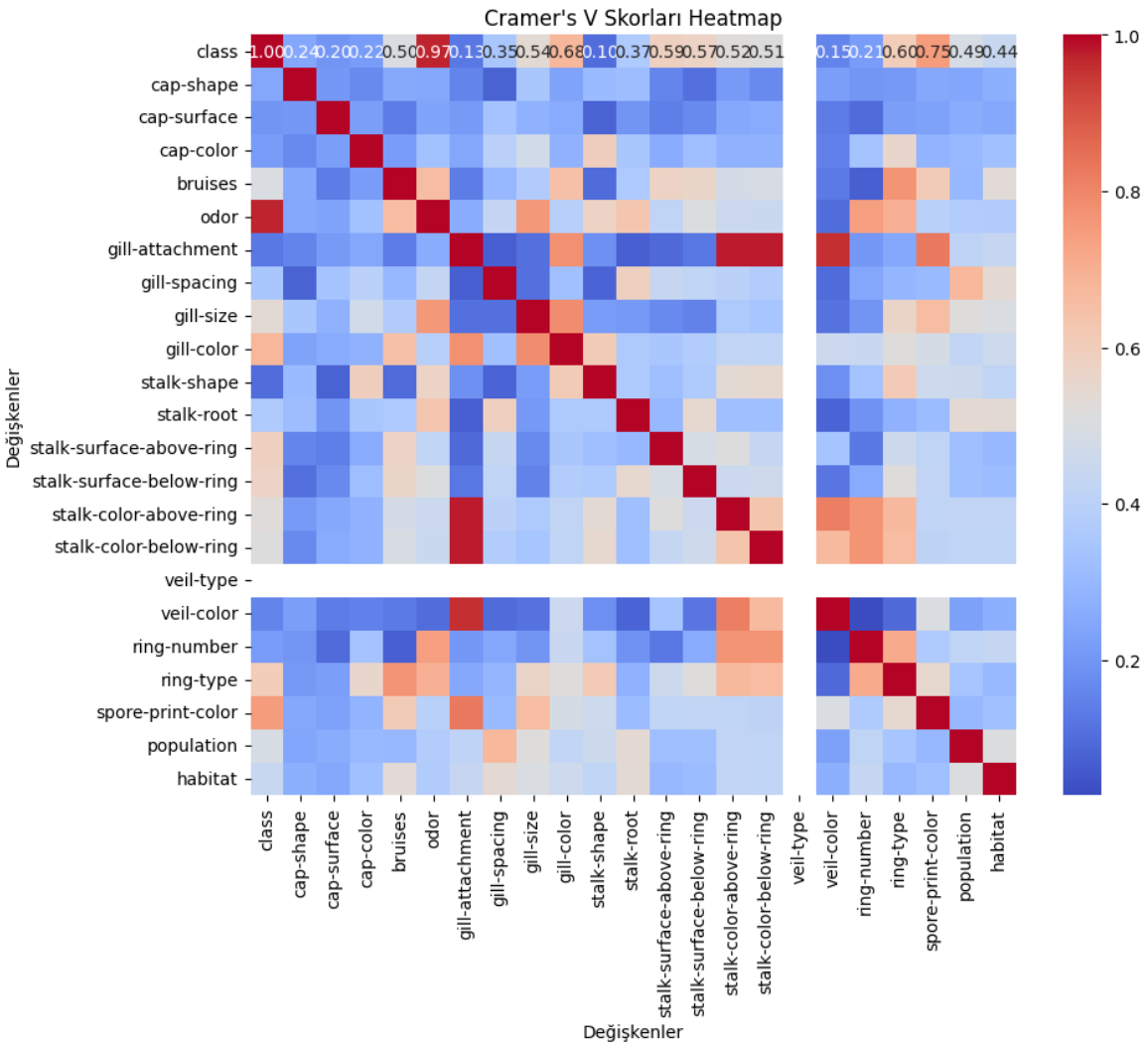
Değişkenlerin kaç etiket içerdiğini gösteren grafiğimiz yukarıdaki gibidir.



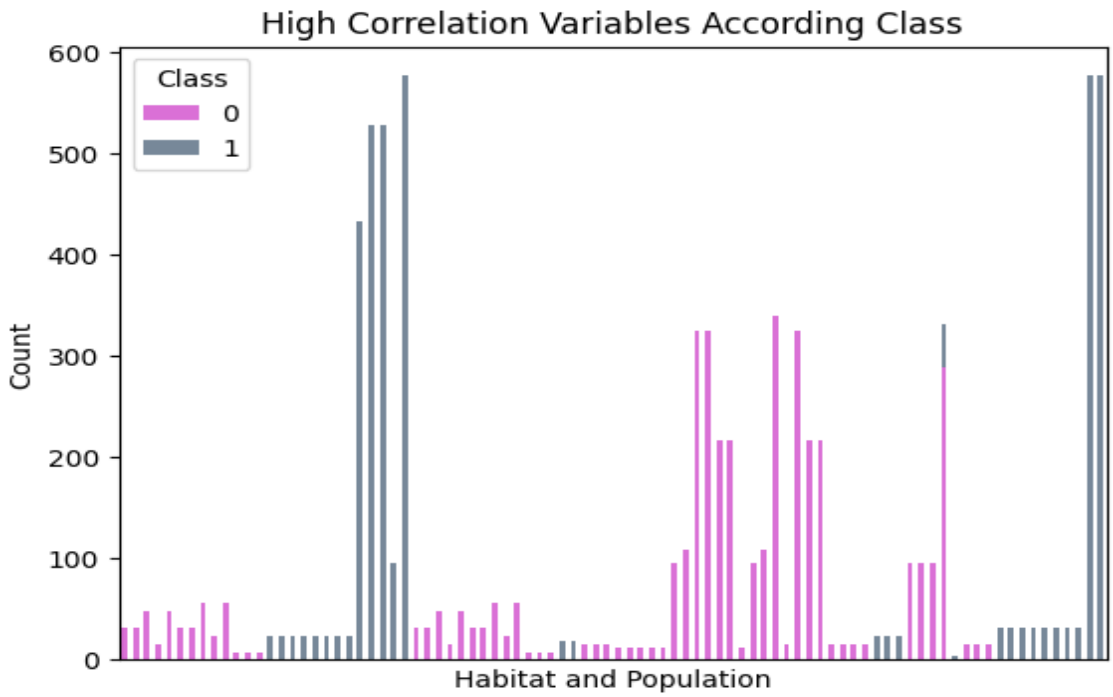
Yukarıda bahsettiğimiz gibi bazı değişkenler homojen dağılmaktadır. Bunların verimizde yaratacağı etkileri modellemelerde ve bazı grafiklerde göreceğiz.

Bu grafiğimizde gördüğümüz gibi bazı değişkenlerin değerleri neredeyse ya zehirli, ya yenilebilir şekilde dağılmakta. Örneğin odor'u f olan mantarların sadece zehirli n olanların neredeyse yenilebilir olduğu gibi yoğunlaşmalar bulunmaktadır.

Şimdi kategorik değişkenler için kullanılan korelasyon analizi olan Cramer’s V metodu ile ısı grafiğini inceleyelim.



Bağımlı değişkenimiz olan class ile kuvvetli korelasyonları olan değişkenlere baktığımızda, odor 0.97 spore-print-color 0.75 , gill-color 0.68 ile dikkat çekmektedir. Şimdi verimizi bu değişkenlere göre gruplayıp, zehirli zehirsiz olma dağılımlarına bakalım. (Bu analiz ilgili platformlardan alınmamıştır.)



Sadece bu değişkenlere gruplayıp baktığımız zaman neredeyse %100 e yakın mantarların zehirli olup olmadığını görmek mümkün, verimizde bunun gibi bir sürü örüntü bulunup, mantarlarımızın class'ını tahmin ettiğimiz modellemelerin işini oldukça kolaylaştırdığından %100 e yakın doğruluk oranları göreceğiz.

Modelleme

Önce modellemeye hazır hale getirmek için etiketleyelim.

Label Encoding

```
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df = df.apply(label_encoder.fit_transform)
```

```
df.head()
```

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	...	stalk-surface-below-ring	stalk-color-above-ring	stalk-color-below-ring	veil-type	veil-color	ring-number	ring-type	spore-print-color	population
0	1	5	2	4	1	6	1	0	1	4	...	2	7	7	0	2	1	4	2	3
1	0	5	2	9	1	0	1	0	0	4	...	2	7	7	0	2	1	4	3	2
2	0	0	2	8	1	3	1	0	0	5	...	2	7	7	0	2	1	4	3	2
3	1	5	3	8	1	6	1	0	1	5	...	2	7	7	0	2	1	4	2	3
4	0	5	2	3	0	5	1	1	0	4	...	2	7	7	0	2	1	0	3	0

5 rows × 23 columns

Şimdi sınıflandırma modellememizi seçmek adına aday classifier ve ensemble algoritmalarımızı deneyip doğruluklarına bakalım.

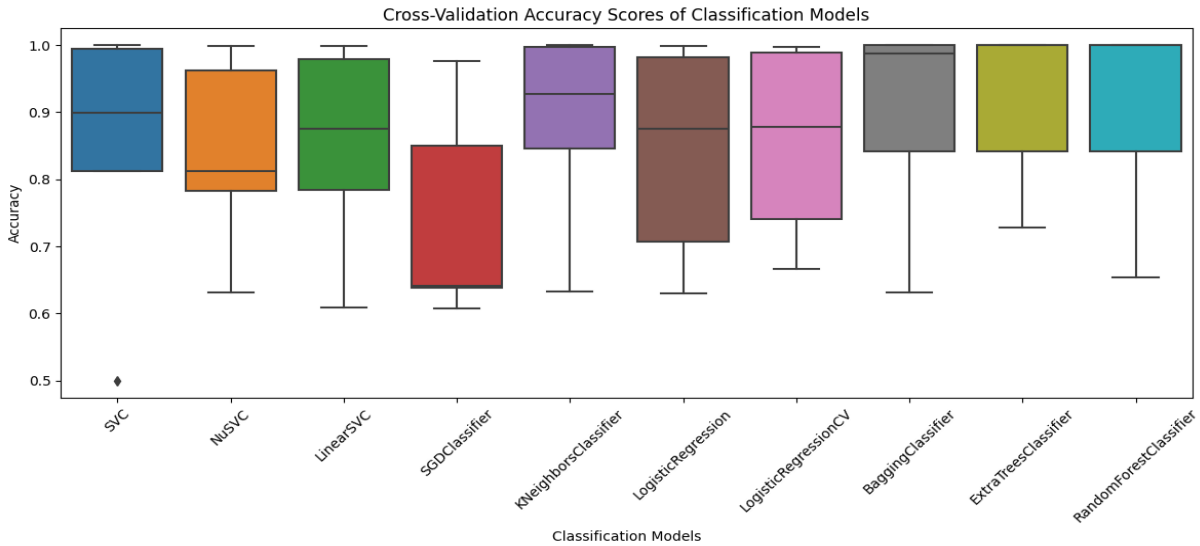
```python
from sklearn.metrics import f1_score
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC, NuSVC, SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.linear_model import LogisticRegressionCV, LogisticRegression, SGDClassifier
from sklearn.ensemble import BaggingClassifier, ExtraTreesClassifier, RandomForestClassifier


models = [
    SVC(gamma='auto'), NuSVC(gamma='auto'), LinearSVC(),
    SGDClassifier(max_iter=100, tol=1e-3), KNeighborsClassifier(),
    LogisticRegression(solver='lbfgs'), LogisticRegressionCV(cv=3),
    BaggingClassifier(), ExtraTreesClassifier(n_estimators=300),
    RandomForestClassifier(n_estimators=300)
]


def score_model(X, y, estimator, **kwargs):
    """
    Test various estimators.
    """
    y = LabelEncoder().fit_transform(y)
    model = Pipeline([
        ('one_hot_encoder', OneHotEncoder()),
        ('estimator', estimator)
    ])

    # Instantiate the classification model and visualizer
    model.fit(X, y, **kwargs)

    expected  = y
    predicted = model.predict(X)

    # Compute and return F1 (harmonic mean of precision and recall)
    print("{}: {}".format(estimator.__class__.__name__, f1_score(expected, predicted)))

for model in models:
    score_model(X, y, model)
```

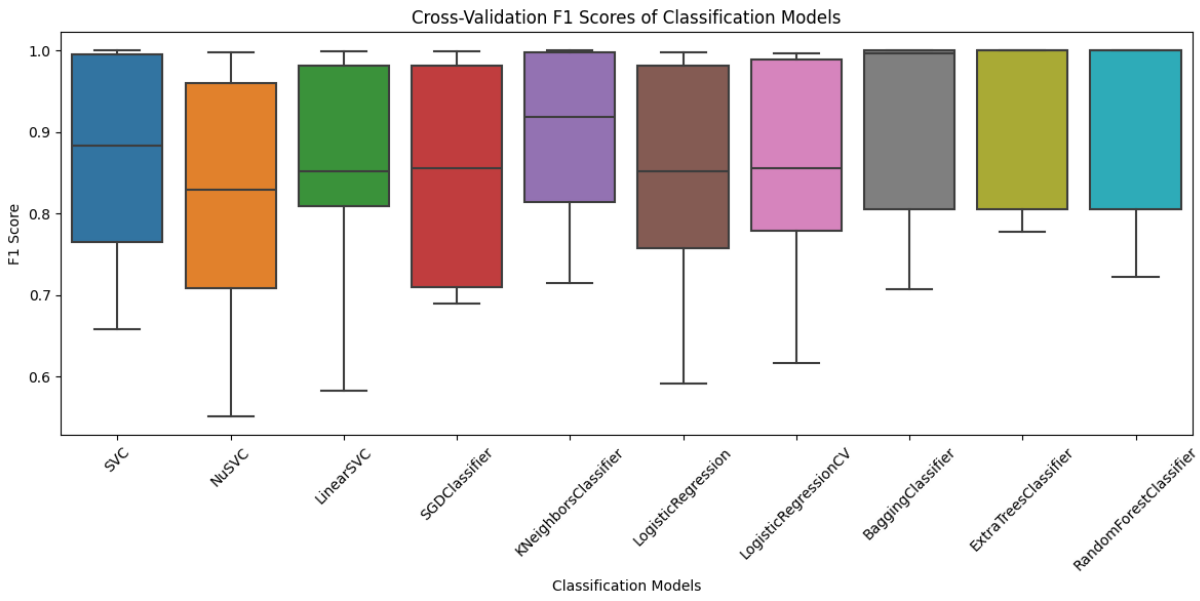
```
SVC: 0.9989775051124744
NuSVC: 0.8976640711902113
LinearSVC: 1.0
SGDClassifier: 1.0
```

```
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\svm\_classes.py:31: FutureWarning: The default value of `dual` will change
from `True` to ``'auto'`` in 1.5. Set the value of `dual` explicitly to suppress the warning.
  warnings.warn(
```

```
KNeighborsClassifier: 1.0
LogisticRegression: 1.0
LogisticRegressionCV: 1.0
BaggingClassifier: 1.0
ExtraTreesClassifier: 1.0
RandomForestClassifier: 1.0
```

Yüksek gelen doğruluk oranlarının ardından cross validation ile doğruluk oranlarına bakıp daha objektif bir karşılaştırma yapalım.





Modellerin f1 skorlarına baktığımızda K Neighbors, Linear SVC, Logistic Regression öne çıkmaktadır.

Biz projemizde K Neighbors, Logistic Regresyon ve ensemble metod olarak Random Forest, ek olarak Denetimsiz öğrenme metodu olan KMeans kümeleme algoritmasıyla boyut indirgeme tahmin ve verimizin karşılaştırmalı görselleştirmesini yapacağız.

K Neighbors

K Neighbors Classifier

```
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# KNN sınıflandırıcıyı oluşturma
knn = KNeighborsClassifier(n_neighbors=5)

# Çapraz doğrulama yapma
scores = cross_val_score(knn, X, y, cv=5)  # cv parametresi K için kullanılacak kat sayıyı belirtir

# Sonuçları görüntüleme
print("Çapraz Doğrulama Skorları:", scores)
print("Ortalama Doğruluk:", scores.mean())
```

```
Çapraz Doğrulama Skorları: [0.84615385 0.99753846 0.92738462 1.         0.63300493]
Ortalama Doğruluk: 0.8808163698370594
```

Verimizi cross validation ile modellediğimiz zaman doğruluk oranları ve ortalamalarını veren kod ile çıktısı yukardadır.Burada %63 lik kısım ortalamayı oldukça düşürmektedir. Model verinin o kısmında yeterince iyi çalışmamaktadır.

Hiperparametre optimizasyonu

```python
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_neighbors': range(1, 20),  # Komşu sayısı
    'weights': ['uniform', 'distance'],  # Ağırlık fonksiyonu
    'metric': ['euclidean', 'manhattan', 'chebyshev']  # Mesafe metriği
}

# Grid arama ile hiperparametre optimizasyonu yapma
grid_search = GridSearchCV(knn, param_grid, cv=5)
grid_search.fit(X, y)

# En iyi hiperparametreleri ve en iyi skoru görüntüleme
print("En İyi Hiperparametreler:", grid_search.best_params_)
print("En İyi Skor:", grid_search.best_score_)
```

```
En İyi Hiperparametreler: {'metric': 'manhattan', 'n_neighbors': 2, 'weights': 'uniform'}
En İyi Skor: 0.9258835922697992
```

```python
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# KNN sınıflandırıcıyı oluşturma
knn = KNeighborsClassifier(n_neighbors=2,metric="manhattan",weights="uniform")

# Çapraz doğrulama yapma
scores = cross_val_score(knn, X, y, cv=5)  # cv parametresi K için kullanılacak kat sayıyı belirtir

# Sonuçları görüntüleme
print("Çapraz Doğrulama Skorları:", scores)
print("Ortalama Doğruluk:", scores.mean())
```

```
Çapraz Doğrulama Skorları: [0.84246154 1.         0.98276923 1.         0.80418719]
Ortalama Doğruluk: 0.9258835922697992
```

K Neighbors'un parametrelerini deneyip optimal parametreleri bulduk, uzaklık metriği olarak manhattan uzaklığı, komşu sayısı olarak 2 verimizin ağırlığı da uniform olarak belirlenmiştir.

Çapraz doğrulama ile arttığı %4 arttığı, optimizasyonun ne kadar önemli olduğu anlaşılmıştır.

Lojistik Regresyon

Raw Model

```python
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=21)

# Logistic Regresyon modelini oluştur ve eğit
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Test seti üzerinde modelin performansını değerlendir
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9649230769230769
```

İlk modellememiz sonucu %96,49 doğruluk oranı mevcuttur.

Raw Model's Cross Validations

```
cv_scores = cross_val_score(log_reg, X, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
```

Cross validation yaptığımızda ortalama 84'e yakın gelmekte.

Hiperparametre optimizasyonu

Raw Model's Hyperparameter Optimization

```
from sklearn.model_selection import GridSearchCV

# Grid Search için parametrelerin belirlenmesi
param_grid = {'C': [0.1, 1, 10, 100], 'penalty': ['l1', 'l2']}

# Grid Search ile Logistic Regresyon modelini optimize et
grid_search = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid_search.fit(X_train, y_train)

# En iyi parametreleri ve en iyi skoru yazdır
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

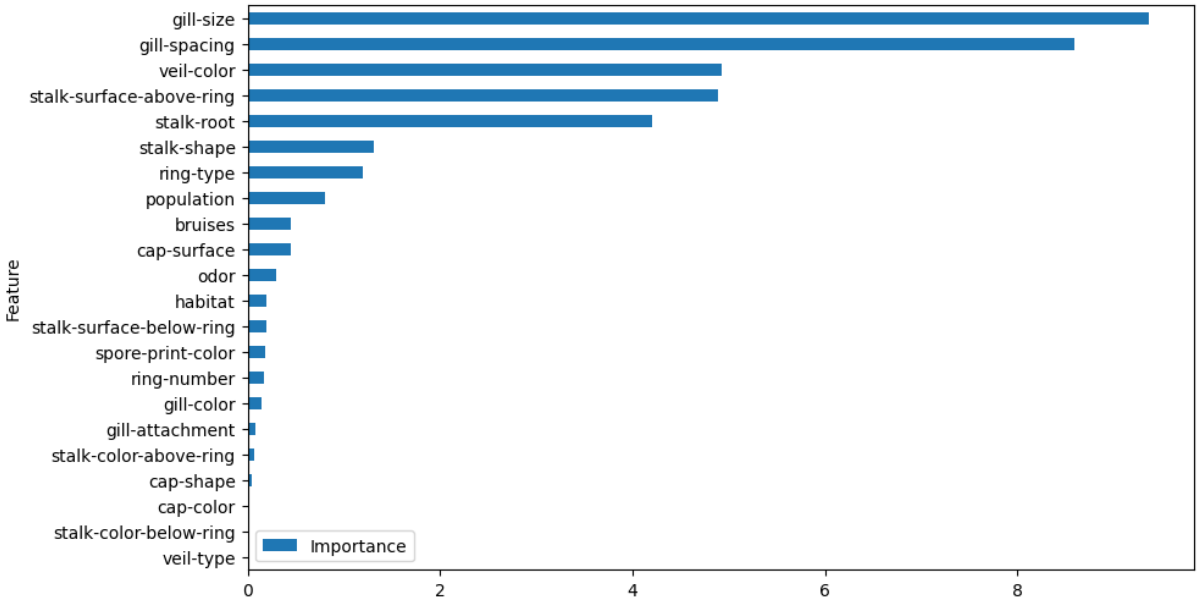
```
    raise ValueError(
ValueError: Solver lbfgs supports only 'l2' or None penalties, got l1 penalty.

  warnings.warn(some_fits_failed_message, FitFailedWarning)
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_search.py:1051: UserWarning: One or more of the test s
cores are non-finite: [       nan 0.93429786        nan 0.94783798        nan 0.96276343
        nan 0.96599491]
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:469: ConvergenceWarning: lbfgs failed to conv
erge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(

Best Parameters: {'C': 100, 'penalty': 'l2'}
Best Score: 0.9659949073251614
```

Parametreler denendiği zaman , doğruluğun %97 ye yaklaştığı görülmektedir.

Lojistik Regresyon ile değişken seçimi

Logistic Regression Feature Importances

```
log_reg.fit(X_train, y_train)


coefficients = log_reg.coef_[0]

feature_importance = pd.DataFrame({'Feature': X.columns, 'Importance': np.abs(coefficients)})
feature_importance = feature_importance.sort_values('Importance', ascending=True)
feature_importance.plot(x='Feature', y='Importance', kind='barh', figsize=(10, 6))
```



Önem sırasına göre aldığımızda, populationa kadar olan değişkenler seçilmiştir.

Model With Selected Features

```
X_log=df[["gill-size","gill-spacing","veil-color","stalk-surface-above-ring","stalk-root","ring-type"]]
X_train, X_test, y_train, y_test = train_test_split(X_log, y, test_size=0.2, random_state=21)

# Logistic Regresyon modelini oluştur ve eğit
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Test seti üzerinde modelin performansını değerlendir
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9538461538461539
```

```
cv_scores = cross_val_score(log_reg, X_log, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
```

```
Cross-Validation Scores: [0.89046154 0.97784615 0.87569231 0.99876923 0.70135468]
Mean CV Score: 0.8888247821144372
```

Seçilen değişkenlerle beraber modelleme doğruluğu ve çapraz doğrulama ile skorlara bakıldığında ortalama %88 doğruluğa ulaşılmıştır.

Hyperparameter Optimization With Selected Features

```
]: from sklearn.model_selection import GridSearchCV

# Grid Search için parametrelerin belirlenmesi
param_grid = {'C': [0.1, 1, 10, 100], 'penalty': ['l1', 'l2']}

# Grid Search ile Logistic Regresyon modelini optimize et
grid_search = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid_search.fit(X_train, y_train)

# En iyi parametreleri ve en iyi skoru yazdır
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
```

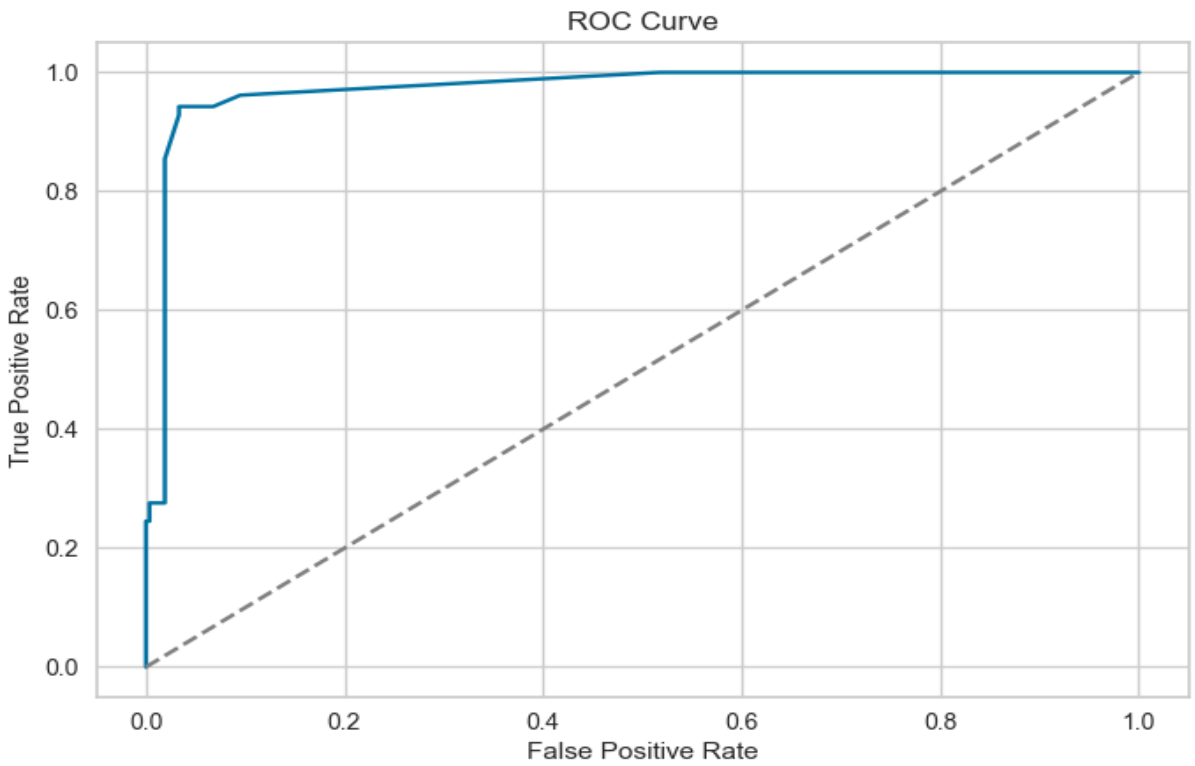
```
Best Parameters: {'C': 10, 'penalty': 'l2'}
Best Score: 0.9361437792384674
```

```
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:547: FitFailedWarning:
20 fits failed out of a total of 40.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

Below are more details about the failures:
--------------------------------------------------------------------------------
20 fits failed with the following error:
Traceback (most recent call last):
  File "C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py", line 895, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\base.py", line 1474, in wrapper
    return fit_method(estimator, *args, **kwargs)
  File "C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py", line 1172, in fit
    solver = _check_solver(self.solver, self.penalty, self.dual)
  File "C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py", line 67, in _check_solver
    raise ValueError(
```

```
]: cv_scores = cross_val_score(LogisticRegression(C= 10, penalty='l2'), X_log, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
```

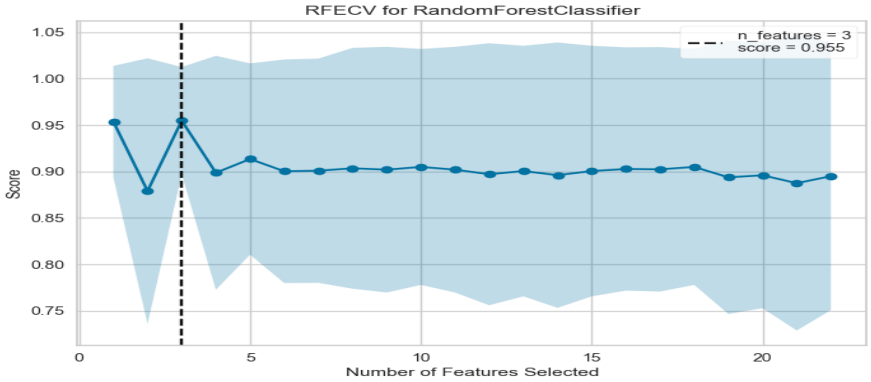
```
Cross-Validation Scores: [0.89046154 0.97784615 0.87569231 0.99876923 0.86330049]
Mean CV Score: 0.9212139446760135
```

Hiper parametre optimizasyonunun ardından, gerekli parametre ve değişkenler ile ortalama cv doğruluğunun %84 den %92 ye çıktığı görülmüştür.



Roc curvemize baktığımızda alanımız 1 br^2 oldukça yakın gelmektedir. Roc skorumuz 0.973 dır.

ENSEMBLE METHOD (RANDOM FOREST)



Verimizde önce seçeceğimiz kaç tane değişken seçmemiz gerektiğini Recursive feature elimination metoduyla bulduk (İnternette ilgili kaynaklarda görülmemiştir.)

Best 3 Feature Selection According Random Forrest

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from mlxtend.feature_selection import SequentialFeatureSelector as SFS
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model=RandomForestClassifier(n_estimators=100, random_state=42)
sfs = SFS(model,
          k_features=3,
          forward=True,
          floating=False,
          scoring='accuracy',
          cv=5)

sfs.fit(X_train, y_train)
selected_features_idx = sfs.k_feature_idx_
selected_features_names = X.columns[list(selected_features_idx)]
print("En iyi 3 özellik:", selected_features_names)
```

```
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(
C:\Users\Batuhan\anaconda3\lib\site-packages\sklearn\model_selection\_validation.py:73: FutureWarning: `fit_params` is depre
cated and will be removed in version 1.6. Pass parameters via `params` instead.
  warnings.warn(

En iyi 3 özellik: Index(['cap-color', 'odor', 'spore-print-color'], dtype='object')
```

Sequential Feature Selector ile de en iyi 3 değişkeni seçiyoruz.

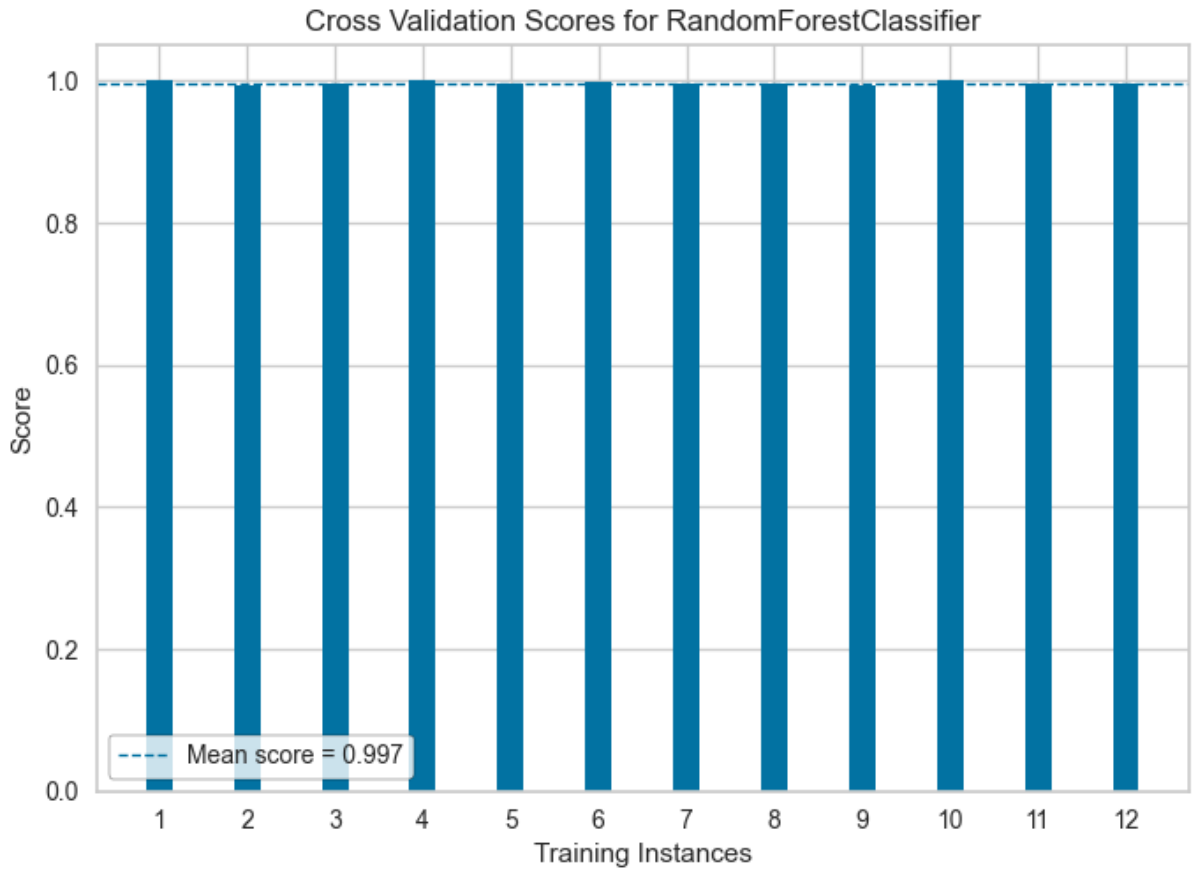
Cross Validation with new variables

```
from sklearn.model_selection import GridSearchCV

# Hiperparametre aralıkları
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# GridSearchCV kullanarak optimizasyon
rf = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=3, scoring='accuracy')
grid_search.fit(X_train, y_train)

print("En iyi parametreler:", grid_search.best_params_)
print("En iyi doğruluk:", grid_search.best_score_)
```

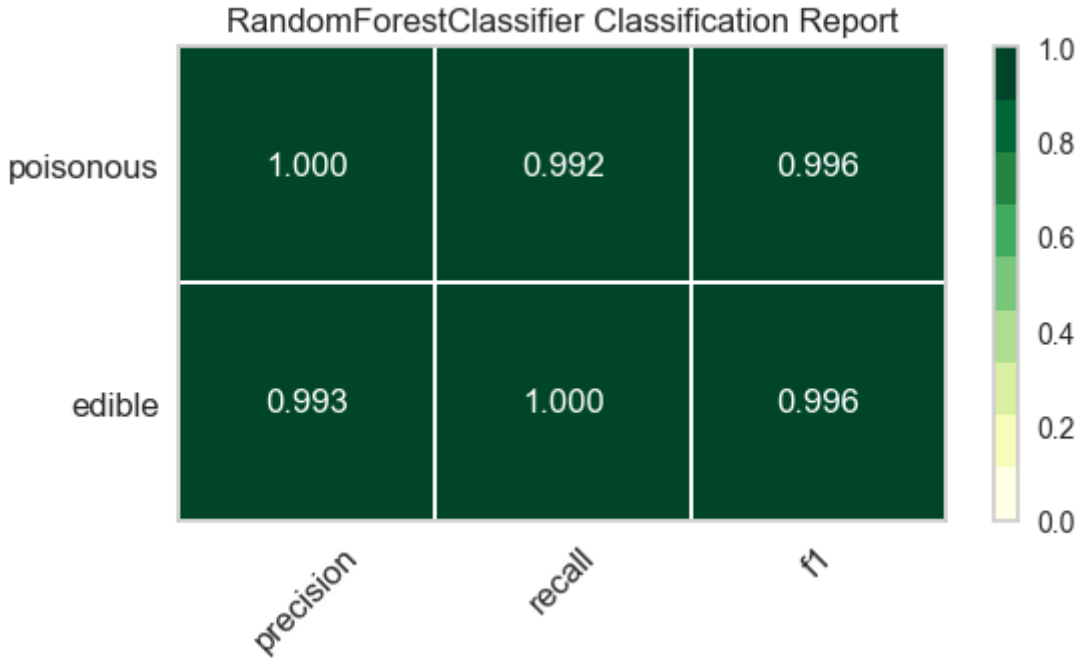
```
En iyi parametreler: {'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 50}
En iyi doğruluk: 0.9972304140154303
```

```
X=df[['cap-color', 'odor', 'spore-print-color']]
y=df["class"]
# Create a cross-validation strategy
cv = StratifiedKFold(n_splits=12, shuffle=True, random_state=42)
visualizer = CVScores(RandomForestClassifier(max_depth= None, min_samples_leaf= 1, min_samples_split= 2, n_estimators= 50), cv=cv

visualizer.fit(X, y)
visualizer.show()
```

Ardından cross validation optimal parametrele ile neredeyse %100 e yakın doğruluk oranları elde ediliyor.

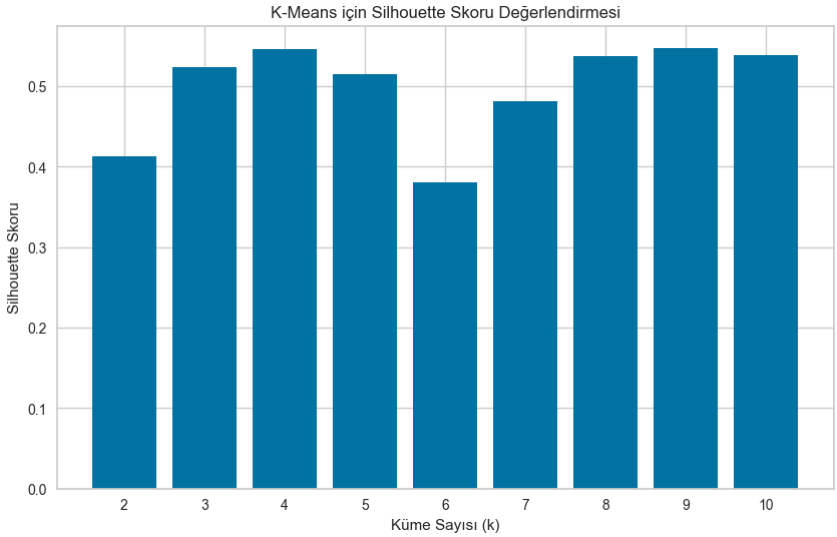


Karar ağacının görselleştirmesi



Yenilebilir mantarların %99.3 luk kısmını doğru tahmin etmekteyiz, f1 scorelarımız da 1 e çok yakın olduğu için modelimizi anlamlı ve %100 e yakın olduğunu kabul edebiliriz.

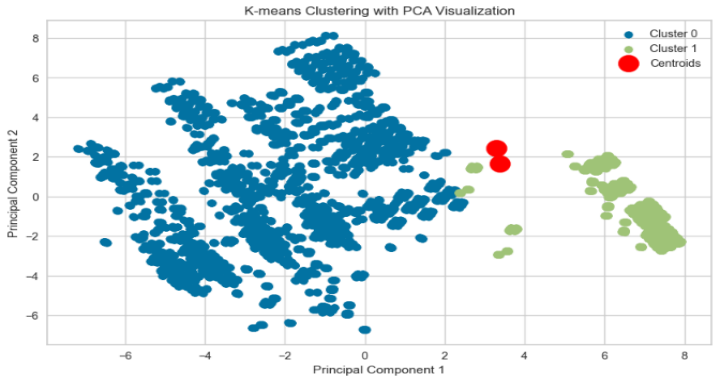
K Means ve PCA ile görselleştirme



Küme sayımız olan 2 için yaklaşık %40 doğruluk oranı görünmektedir.

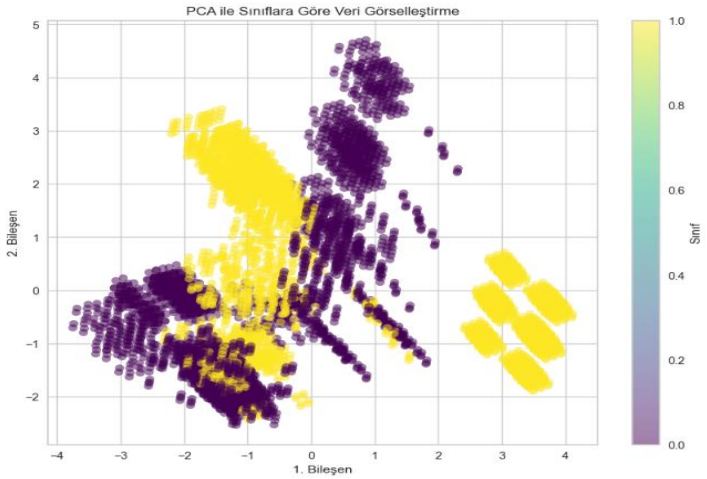
K Means(Unsupervised) Clustering

```python
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Veriyi özellikler (X) ve etiketler (y) olarak ayırma
X = df.drop('class', axis=1)
y = df['class']

# K-means modelini oluşturma ve eğitme
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)

# PCA ile boyut indirgeme
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Küme merkezlerini ve küme etiketlerini olarak görselleştirme
plt.figure(figsize=(10, 6))

for i in range(2):
    plt.scatter(X_pca[kmeans.labels_ == i, 0], X_pca[kmeans.labels_ == i, 1], label=f'Cluster {i}')

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='red', label='Centroids')
plt.title('K-means Clustering with PCA Visualization')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



Verimizi k ortalamalar algoritmasına göre sınıflandırıp, boyut 22 olan boyutumuzu 2 boyuta indirgeyip, tahmin edilen sınıfları görselleştirdik.

```python
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Önce verileri standartlaştırma
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA modelini oluşturma ve uygulama
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Sınıflara göre renklendirilmiş scatter plot çizimi
plt.figure(figsize=(10, 8))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', alpha=0.5)
plt.title('PCA ile Sınıflara Göre Veri Görselleştirme')
plt.xlabel('1. Bileşen')
plt.ylabel('2. Bileşen')
plt.colorbar(label='Sınıf')
plt.grid(True)
plt.show()
```



Ancak asıl verimizi görselleştirdiğimizde büyük bir kısmı yanlış sınıflandırdığımız görülmektedir.

Tüm Kodlar

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC, NuSVC, SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegressionCV, LogisticRegression, SGDClassifier
from sklearn.ensemble import BaggingClassifier, ExtraTreesClassifier, RandomForestClassifier
```

```
from yellowbrick.classifier import ClassificationReport
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.cluster import KMeans

df=pd.read_csv("mushrooms.csv")
df.head()
df.info()
df.dtypes
df.isna().sum()
df.describe()
plt.figure(figsize=(16, 20))
for i, column in enumerate(df.columns[1:], 1):
    plt.subplot(6, 4, i)
    sns.countplot(x=column, data=df, palette="viridis")
    plt.title(f"{column} Countplot")
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
categorical_counts = df.apply(lambda col: col.value_counts().shape[0])

# Sütun grafiği oluşturalım
plt.figure(figsize=(10, 6))
categorical_counts.plot(kind='bar', color='skyblue')
plt.title('Her Sütundaki Kategorik Değişkenlerin Sayısı')
plt.xlabel('Sütunlar')
plt.ylabel('Değer Sayısı')
plt.xticks(rotation=45)
plt.tight_layout()
```

```
plt.show()
def cramers_v_matrix(data):
    categorical_vars = data.select_dtypes(include=['object', 'category']).columns
    n_vars = len(categorical_vars)
    matrix = np.zeros((n_vars, n_vars))

    for i, var1 in enumerate(categorical_vars):
        for j, var2 in enumerate(categorical_vars):
            if i <= j:
                confusion_matrix = pd.crosstab(data[var1], data[var2])
                chi2 = chi2_contingency(confusion_matrix)[0]
                n = confusion_matrix.sum().sum()
                phi2 = chi2 / n
                r, k = confusion_matrix.shape
                phi2corr = max(0, phi2 - ((k - 1) * (r - 1)) / (n - 1))
                rcorr = r - ((r - 1) ** 2) / (n - 1)
                kcorr = k - ((k - 1) ** 2) / (n - 1)
                matrix[i, j] = np.sqrt(phi2corr / min((kcorr - 1), (rcorr - 1)))
                matrix[j, i] = matrix[i, j]

    return pd.DataFrame(matrix, index=categorical_vars, columns=categorical_vars)

# Örnek kullanım
# Veri setiniz 'data' adlı bir DataFrame ise:
# data = pd.read_csv("veri.csv")
cramer_v_results = cramers_v_matrix(df)
plt.figure(figsize=(10, 8))
sns.heatmap(cramer_v_results, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Cramer's V Skorları Heatmap")
plt.xlabel("Değişkenler")
plt.ylabel("Değişkenler")
```

```
plt.show()
# Grafikleri yan yana sığdırmak için subplotlar oluşturalım
fig, axes = plt.subplots(nrows=6, ncols=4, figsize=(20, 20))

# Her bir değişken için gruplanmış verileri çizelim
for i, ax in enumerate(axes.flatten()):
    if i < len(df.columns):
        grouped = df.groupby([df.columns[i], 'class']).size().unstack(fill_value=0)
        colors = ["#DA70D6", "#778899"]
        grouped.plot(kind='bar', color=colors, stacked=True, ax=ax)
        ax.set_title('{}'.format(df.columns[i]))
        ax.set_xlabel('')
        ax.set_ylabel('Count')
        ax.legend(title='Class')

# Eksenlerin üst üste gelmemesi için layout ayarlayalım
plt.tight_layout()
plt.show()
grouped = df.groupby(['odor', 'spore-print-color', 'gill-color','class']).size().unstack(fill_value=0)
colors = ["#DA70D6", "#778899"]

grouped.plot(kind='bar',color=colors, stacked=True)
plt.title('High Correlation Variables According Class')
plt.xlabel('Habitat and Population')
plt.ylabel('Count')
plt.xticks([])  # X ekseni etiketlerini kaldırma
plt.legend(title='Class')
plt.show()
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df = df.apply(label_encoder.fit_transform)
```

```
X = df.drop('class', axis=1)  # 'class' sütununu hedef değişken olarak kabul ediyoruz
y = df['class']
models = [
    SVC(gamma='auto'), NuSVC(gamma='auto'), LinearSVC(),
    SGDClassifier(max_iter=100, tol=1e-3), KNeighborsClassifier(),
    LogisticRegression(solver='lbfgs'), LogisticRegressionCV(cv=3),
    BaggingClassifier(), ExtraTreesClassifier(n_estimators=300),
    RandomForestClassifier(n_estimators=300)
]



def score_model(X, y, estimator, **kwargs):
    """
    Test various estimators.
    """
    y = LabelEncoder().fit_transform(y)
    model = Pipeline([
        ('one_hot_encoder', OneHotEncoder()),
        ('estimator', estimator)
    ])

    # Instantiate the classification model and visualizer
    model.fit(X, y, **kwargs)

    expected  = y
    predicted = model.predict(X)

    # Compute and return F1 (harmonic mean of precision and recall)
    print("{}: {}".format(estimator.__class__.__name__, f1_score(expected, predicted)))

for model in models:
```

```
    score_model(X, y, model)

models = [
    ("SVC", SVC()),
    ("NuSVC", NuSVC()),
    ("LinearSVC", LinearSVC()),
    ("SGDClassifier", SGDClassifier()),
    ("KNeighborsClassifier", KNeighborsClassifier()),
    ("LogisticRegression", LogisticRegression()),
    ("LogisticRegressionCV", LogisticRegressionCV(cv=3)),
    ("BaggingClassifier", BaggingClassifier()),
    ("ExtraTreesClassifier", ExtraTreesClassifier()),
    ("RandomForestClassifier", RandomForestClassifier())
]

# Modelleri değerlendir ve çapraz doğrulama skorlarını topla
cv_results = []
model_names = []
for name, model in models:
    scores = cross_val_score(model, X, y, scoring='f1', cv=5)
    cv_results.extend(scores)
    model_names.extend([name] * len(scores))

# Skorları içeren bir veri çerçevesi oluştur
df_scores = pd.DataFrame({"Model": model_names, "F1 Score": cv_results})

# Kutu grafiğiyle görselleştir
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_scores, x="Model", y="F1 Score")
plt.title('Cross-Validation F1 Scores of Classification Models')
plt.xlabel('Classification Models')
```

```
plt.ylabel('F1 Score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
from sklearn.model_selection import cross_val_score

models = [
    ("SVC", SVC()),
    ("NuSVC", NuSVC()),
    ("LinearSVC", LinearSVC()),
    ("SGDClassifier", SGDClassifier()),
    ("KNeighborsClassifier", KNeighborsClassifier()),
    ("LogisticRegression", LogisticRegression()),
    ("LogisticRegressionCV", LogisticRegressionCV(cv=3)),
    ("BaggingClassifier", BaggingClassifier()),
    ("ExtraTreesClassifier", ExtraTreesClassifier()),
    ("RandomForestClassifier", RandomForestClassifier())
]

# Modelleri değerlendir ve çapraz doğrulama skorlarını topla
cv_results = []
model_names = []
for name, model in models:
    scores = cross_val_score(model, X, y, scoring='accuracy', cv=5)
    cv_results.extend(scores)
    model_names.extend([name] * len(scores))

# Skorları içeren bir veri çerçevesi oluştur
df_scores = pd.DataFrame({"Model": model_names, "Accuracy": cv_results})

# Kutu grafiğiyle görselleştir
```

```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df_scores, x="Model", y="Accuracy")
plt.title('Cross-Validation Accuracy Scores of Classification Models')
plt.xlabel('Classification Models')
plt.ylabel('Accuracy')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# KNN sınıflandırıcıyı oluşturma
knn = KNeighborsClassifier(n_neighbors=5)

# Çapraz doğrulama yapma
scores = cross_val_score(knn, X, y, cv=5)  # cv parametresi K için kullanılacak kat sayıyı belirtir

# Sonuçları görüntüleme
print("Çapraz Doğrulama Skorları:", scores)
print("Ortalama Doğruluk:", scores.mean())
from sklearn.model_selection import GridSearchCV
param_grid = {
    'n_neighbors': range(1, 20),  # Komşu sayısı
    'weights': ['uniform', 'distance'],  # Ağırlık fonksiyonu
    'metric': ['euclidean', 'manhattan', 'chebyshev']  # Mesafe metriği
}

# Grid arama ile hiperparametre optimizasyonu yapma
grid_search = GridSearchCV(knn, param_grid, cv=5)
grid_search.fit(X, y)
```

```
# En iyi hiperparametreleri ve en iyi skoru görüntüleme
print("En İyi Hiperparametreler:", grid_search.best_params_)
print("En İyi Skor:", grid_search.best_score_)
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# KNN sınıflandırıcıyı oluşturma
knn = KNeighborsClassifier(n_neighbors=2,metric="manhattan",weights="uniform")

# Çapraz doğrulama yapma
scores = cross_val_score(knn, X, y, cv=5)  # cv parametresi K için kullanılacak kat sayıyı belirtir

# Sonuçları görüntüleme
print("Çapraz Doğrulama Skorları:", scores)
print("Ortalama Doğruluk:", scores.mean())
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=21)

# Logistic Regresyon modelini oluştur ve eğit
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Test seti üzerinde modelin performansını değerlendir
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
cv_scores = cross_val_score(log_reg, X, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
from sklearn.model_selection import GridSearchCV

# Grid Search için parametrelerin belirlenmesi
```

```
param_grid = {'C': [0.1, 1, 10, 100], 'penalty': ['l1', 'l2']}

# Grid Search ile Logistic Regresyon modelini optimize et
grid_search = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid_search.fit(X_train, y_train)

# En iyi parametreleri ve en iyi skoru yazdır
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
log_reg.fit(X_train, y_train)


coefficients = log_reg.coef_[0]

feature_importance = pd.DataFrame({'Feature': X.columns, 'Importance': np.abs(coefficients)})
feature_importance = feature_importance.sort_values('Importance', ascending=True)
feature_importance.plot(x='Feature', y='Importance', kind='barh', figsize=(10, 6))
X_log=df[["gill-size","gill-spacing","veil-color","stalk-surface-above-ring","stalk-root","ring-type"]]
X_train, X_test, y_train, y_test = train_test_split(X_log, y, test_size=0.2, random_state=21)

# Logistic Regresyon modelini oluştur ve eğit
log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

# Test seti üzerinde modelin performansını değerlendir
y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
cv_scores = cross_val_score(log_reg, X_log, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
```

```
# Grid Search için parametrelerin belirlenmesi
param_grid = {'C': [0.1, 1, 10, 100], 'penalty': ['l1', 'l2']}

# Grid Search ile Logistic Regresyon modelini optimize et
grid_search = GridSearchCV(LogisticRegression(), param_grid, cv=5)
grid_search.fit(X_train, y_train)

# En iyi parametreleri ve en iyi skoru yazdır
print("Best Parameters:", grid_search.best_params_)
print("Best Score:", grid_search.best_score_)
cv_scores = cross_val_score(LogisticRegression(C= 10, penalty='l2'), X_log, y, cv=5)  # 5 katlı cross-validation
print("Cross-Validation Scores:", cv_scores)
print("Mean CV Score:", cv_scores.mean())
log_reg = LogisticRegression(C= 10, penalty='l2')
log_reg.fit(X_train, y_train)

# Test seti üzerinde tahmin yap
y_pred_proba = log_reg.predict_proba(X_test)[:,1]  # Sadece pozitif sınıfın olasılıklarını al

# ROC eğrisini çizdir
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.plot(fpr, tpr)
plt.plot([0, 1], [0, 1], '--', color='gray')  # Rastgele tahmin çizgisi
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()

# ROC AUC skorunu hesapla
roc_auc = roc_auc_score(y_test, y_pred_proba)
```

```
print("ROC AUC Score:", roc_auc)
cv = StratifiedKFold(5)
visualizer = RFECV(RandomForestClassifier(), cv=cv, scoring='accuracy')

visualizer.fit(X, y)
visualizer.show()
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model=RandomForestClassifier(n_estimators=100, random_state=42)
sfs = SFS(model,
        k_features=3,
        forward=True,
        floating=False,
        scoring='accuracy',
        cv=5)

sfs.fit(X_train, y_train)
selected_features_idx = sfs.k_feature_idx_
selected_features_names = X.columns[list(selected_features_idx)]
print("En iyi 3 özellik:", selected_features_names)
from sklearn.model_selection import GridSearchCV

# Hiperparametre aralıkları
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# GridSearchCV kullanarak optimizasyon
rf = RandomForestClassifier(random_state=42)
```

```
grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=3, scoring='accuracy')
grid_search.fit(X_train, y_train)

print("En iyi parametreler:", grid_search.best_params_)
print("En iyi doğruluk:", grid_search.best_score_)

X=df[['cap-color', 'odor', 'spore-print-color']]
y=df["class"]
# Create a cross-validation strategy
cv = StratifiedKFold(n_splits=12, shuffle=True, random_state=42)
visualizer = CVScores(RandomForestClassifier(max_depth= None, min_samples_leaf= 1,
min_samples_split= 2, n_estimators= 50), cv=cv, scoring='f1_weighted')

visualizer.fit(X, y)
visualizer.show()
selected_features = ['cap-color', 'odor', 'spore-print-color']
X_selected = df[selected_features]
y = df['class']

# One-hot encoding ile veriyi işleme
X_encoded = pd.get_dummies(X_selected)

# Eğitim ve test veri setlerini ayırma
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.2, random_state=42)

# Tek bir ağaç oluşturma ve eğitme
single_tree_model = DecisionTreeClassifier(random_state=42)
single_tree_model.fit(X_train, y_train)

# Karar ağacını çizme
# Karar ağacını çizme
```

```
plt.figure(figsize=(15, 10))

plot_tree(single_tree_model, feature_names=X_encoded.columns,
class_names=y.unique().astype(str), filled=True)

plt.show()

from yellowbrick.classifier import ClassificationReport

pipeline = Pipeline([

    ('estimator', model)

])


# Create the classification report visualizer

visualizer = ClassificationReport(

    pipeline, classes=['edible', 'poisonous'], cmap="YlGn", size=(600, 360)

)


# Fit the visualizer and generate the classification report

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)


# Display the visualizer

visualizer.show()

X = df.drop('class', axis=1)

y = df['class']


# K-means modelini oluşturma ve eğitme

kmeans = KMeans(n_clusters=2, random_state=42)

kmeans.fit(X)


# PCA ile boyut indirgeme

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)
```

```
# Küme merkezlerini ve küme etiketlerini alarak görselleştirme
plt.figure(figsize=(10, 6))

for i in range(2):
    plt.scatter(X_pca[kmeans.labels_ == i, 0], X_pca[kmeans.labels_ == i, 1], label=f'Cluster {i}')

plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=300, c='red',
label='Centroids')
plt.title('K-means Clustering with PCA Visualization')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# PCA modelini oluşturma ve uygulama
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Sınıflara göre renklendirilmiş scatter plot çizimi
plt.figure(figsize=(10, 8))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', alpha=0.5)
plt.title('PCA ile Sınıflara Göre Veri Görselleştirme')
plt.xlabel('1. Bileşen')
plt.ylabel('2. Bileşen')
plt.colorbar(label='Sınıf')
plt.grid(True)
plt.show()
import matplotlib.pyplot as plt
import numpy as np
```

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

k_values = range(2, 11)  # 2'den 10'a kadar küme sayılarını deneyin

# Her bir küme sayısı için Silhouette skorunu hesaplayın
silhouette_scores = []
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    cluster_labels = kmeans.fit_predict(X)
    silhouette_avg = silhouette_score(X, cluster_labels)
    silhouette_scores.append(silhouette_avg)

# Silhouette skorlarını görselleştirin
plt.figure(figsize=(10, 6))
plt.bar(k_values, silhouette_scores)
plt.xlabel('Küme Sayısı (k)')
plt.ylabel('Silhouette Skoru')
plt.title('K-Means için Silhouette Skoru Değerlendirmesi')
plt.xticks(np.arange(min(k_values), max(k_values)+1, 1))
plt.show()
```

Sonuç

Verimiz çok düzgün ayarlanmış olup neredeyse tüm sınıflandırma modellemelerinde %100 e yakın doğruluk oranları verip, gerekli analizlerle bile modelsiz iyi tahminlemeler yapılabilir.Ancak ensemble metodlar daha dengeli tahminler yaptığından Random forest algoritması tercih edilmiştir.

Kaynakça

https://www.kaggle.com/datasets/uciml/mushroom-classification

https://archive.ics.uci.edu/dataset/73/mushroom

https://www.datacamp.com

https://medium.com/analytics-vidhya/building-classification-model-with-python-9bdfc13faa4b

https://github.com/kanchitank/Mushroom-Classification